# Patrón Plantilla (Template Method)

¿Qué especifica el patrón plantilla?

El patrón Template Method es un patrón de diseño de comportamiento que define el esqueleto de un algoritmo en un método de una clase base (método plantilla), delegando la implementación de ciertos pasos específicos a las subclases.

Su propósito central es evitar la duplicación de código.

El patrón plantilla sigue una estructura que se representa de la siguiente manera:

- **Método Plantilla (templateMethod()):** Es el método principal (habitualmente marcado como final para evitar sobrescrituras) que invoca secuencialmente los pasos del algoritmo.
- **Operaciones Abstractas (primitiveOperation()):** Pasos obligatorios que cada subclase debe implementar.
- **Hooks (Ganchos):** Métodos con una implementación por defecto (habitualmente vacía) que las subclases pueden sobrescribir opcionalmente para alterar el comportamiento en puntos clave del algoritmo.

Realizar el Diagrama de Clases.

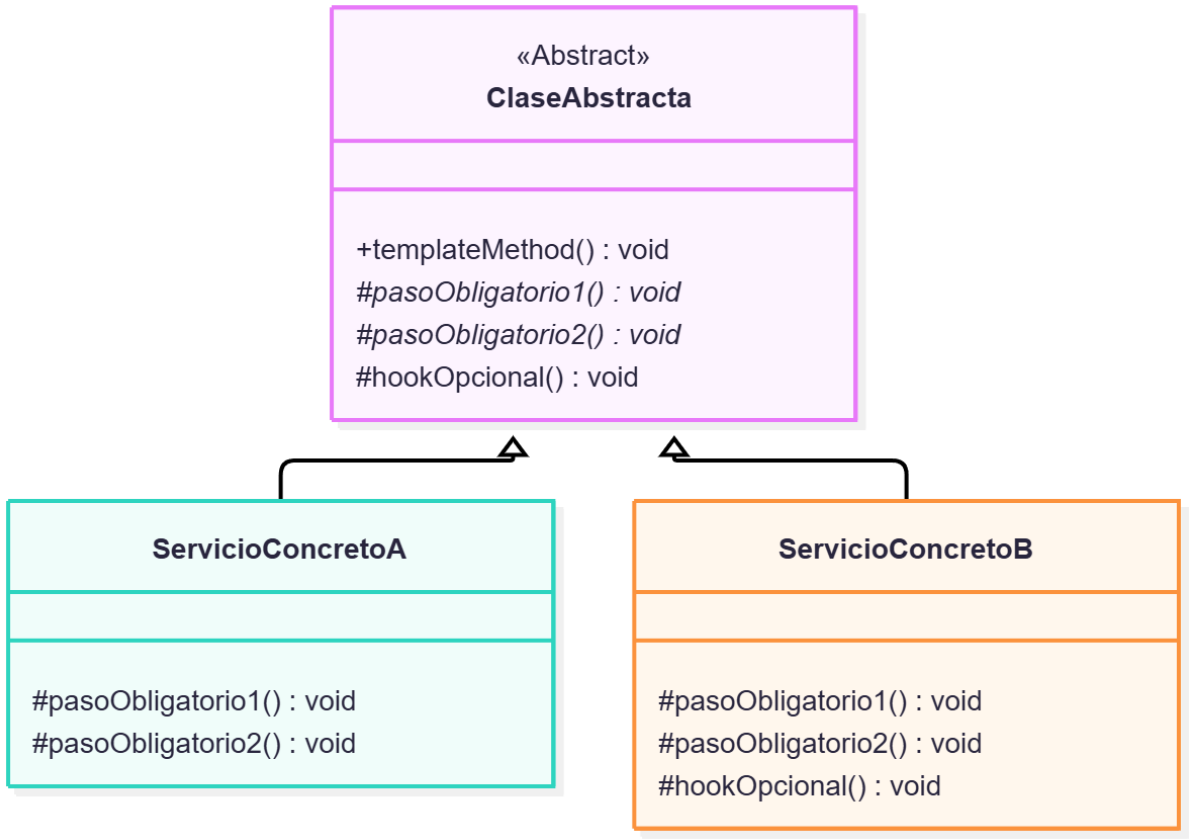



Figura 1: Diagrama de Funcionamiento Patrón Plantilla. Realizado en mermaid.live

¿Cómo se utilizaría el en las clases Controlador y clases de Servicio?

En arquitecturas empresariales (como aplicaciones con Spring Boot/Java EE), el patrón plantilla se utiliza para estandarizar flujos de trabajo repetitivos en la capa web y la capa de negocio:

**En Clases de Servicio**

Estandariza el ciclo de vida de operaciones complejas como la creación o actualización de entidades:

- Validar la petición.
- Aplicar lógica de pre-procesamiento / enriquecimiento de datos.
- Persistir en la base de datos.
- Ejecutar lógica post-procesamiento (p. ej., enviar notificaciones o auditar).

**En Clases Controlador**

- Estandariza la atención de peticiones HTTP:
- Mapear o deserializar el DTO de entrada.
- Invocar la ejecución del servicio correspondiente.
- Formatear y construir la respuesta HTTP estandarizada (ResponseEntity).
- Manejar excepciones comunes del endpoint.

# Java Generics

¿Qué es Java Generics y cuáles son sus ventajas?

Java Generics (introducido en Java 5) es una característica del lenguaje que permite parametrizar tipos en clases, interfaces y métodos. Permite abstraer el tipo de datos sobre el que opera una clase o método.

Ventajas principales

- Seguridad de tipos en tiempo de compilación (Type Safety): Detecta errores de incompatibilidad de tipos durante la compilación en lugar de lanzar una excepción ClassCastException en tiempo de ejecución.
- Eliminación de castings explícitos: Evita la conversión manual e insegura de tipos (por ejemplo, desde Object).
- Reutilización de código: Permite escribir algoritmos genéricos, estructuras de datos y componentes reutilizables sin duplicar lógica por cada tipo de objeto.

¿Cómo podría complementarse con el patrón plantilla para generalizar las clases Controlador y clases de Servicio?

Al combinar Generics con Template Method, se puede construir una jerarquía base reutilizable donde la estructura del algoritmo está definida genéricamente para cualquier entidad o DTO de la aplicación.

```
// 1. SERVICIO BASE GENÉRICO CON TEMPLATE METHOD
public abstract class BaseService<T, ID> {

    // Método Plantilla: define el algoritmo inmutable de creación
    public final T create(T entity) {
        validate(entity);
        preSaveHook(entity);
        T savedEntity = executeSave(entity);
        postSaveHook(savedEntity);
        return savedEntity;
    }
```

```
        // Pasos obligatorios
        protected abstract void validate(T entity);
        protected abstract T executeSave(T entity);

        // Hooks opcionales
        protected void preSaveHook(T entity) { }
        protected void postSaveHook(T entity) { }
    }

    // 2. SERVICIO CONCRETO (Define solo la lógica específica del
dominio)
    public class UsuarioService extends BaseService<Usuario, Long> {

        @Override
        protected void validate(Usuario usuario) {
            if (usuario.getEmail() == null ||
!usuario.getEmail().contains("@")) {
                throw new IllegalArgumentException("Email inválido");
            }
        }

        @Override
        protected Usuario executeSave(Usuario usuario) {
            // Invocación al repositorio específico
            return usuario; // Simulación de persistencia
        }

        @Override
```

```
    protected void preSaveHook(Usuario usuario) {
        // Encriptar contraseña antes de guardar
        usuario.setPassword("HASHED_" + usuario.getPassword());
    }
}

// 3. CONTROLADOR BASE GENÉRICO
public abstract class BaseController<T, ID> {

    protected abstract BaseService<T, ID> getService();

    // Endpoint genérico
    public T handlePostRequest(T entity) {
        // Invocación al flujo del Template Method
        return getService().create(entity);
    }
}
```

# Java Reflection

¿Qué es Java Reflection y cuáles son sus ventajas?

Java Reflection es una API provista por el paquete java.lang.reflect que permite a un programa inspeccionar, examinar y modificar la estructura interna y el comportamiento de clases, interfaces, campos y métodos en tiempo de ejecución (runtime), aun cuando no se conozcan sus nombres en tiempo de compilación.

Ventajas principales

- **Base de frameworks modernos**: Es la piedra angular de frameworks como Spring, Hibernate, Jackson o JUnit para realizar inyección de dependencias, mapeo de tablas a objetos (ORM), serialización de JSON y ejecución de pruebas de forma automática.
- **Inspección de anotaciones**: Permite leer metadatos (@Entity, @Autowired, @Override) en runtime y alterar la ejecución en consecuencia.
- **Desacoplamiento dinámico**: Permite instanciar clases e invocar métodos dinámicamente mediante configuración externa sin acoplamiento estático.

Realizar en código de cómo se accede a los atributos privados de una clase y como se puede invocar un método.

```
import java.lang.reflect.Field;
import java.lang.reflect.Method;

class Persona {
    private String claveSecreta = "1234-ABCD";

    private String generarSaludo(String titulo) {
        return "Hola " + titulo + ", tu clave es: " + claveSecreta;
    }
}

public class DemostracionReflection {
```

```
    public static void main(String[] args) {
        try {
            Persona persona = new Persona();
            Class<?> clase = persona.getClass();

            // ------------------------------------------------------
--------
            // 1. ACCEDER Y MODIFICAR UN ATRIBUTO PRIVADO
            // ------------------------------------------------------
--------
            Field campoClave =
clase.getDeclaredField("claveSecreta");

            // Bypass de los controles de acceso de Java
            campoClave.setAccessible(true);

            // Leer el valor del campo privado
            String valorOriginal = (String) campoClave.get(persona);
            System.out.println("Valor privado original: " +
valorOriginal);

            // Modificar el valor del campo privado
            campoClave.set(persona, "5678-XYZW");
            System.out.println("Valor privado modificado: " +
campoClave.get(persona));

            // ------------------------------------------------------
--------
            // 2. INVOCAR UN MÉTODO PRIVADO CON PARÁMETROS
            // ------------------------------------------------------
--------
```

```
            Method metodoSaludo =
clase.getDeclaredMethod("generarSaludo", String.class);

            // Bypass del control de acceso del método
            metodoSaludo.setAccessible(true);

            // Invocación dinámica pasando la instancia y los
argumentos
            Object resultado = metodoSaludo.invoke(persona,
"Ingeniero");
            System.out.println("Resultado de invocación remota: " +
resultado);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```